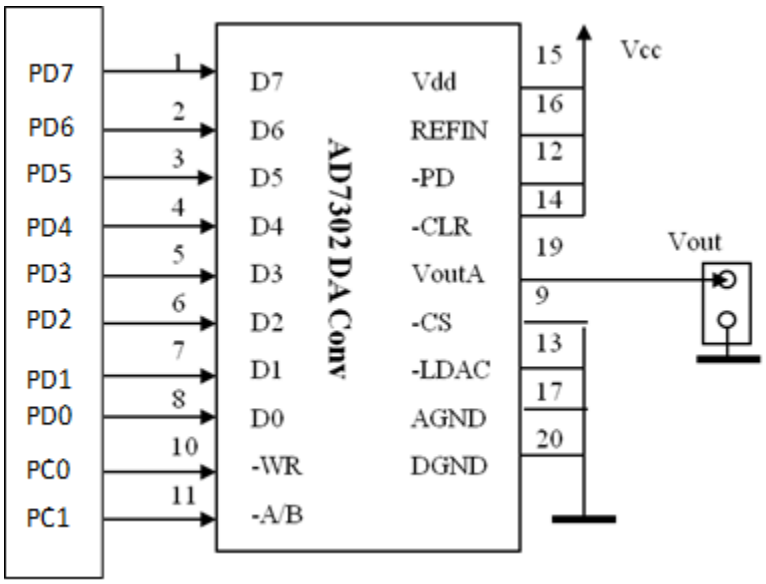
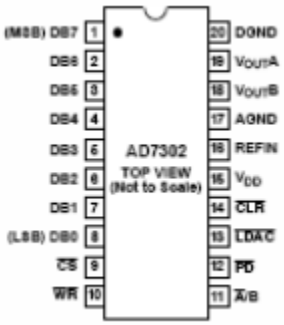
ECE 3218 Lab 04 - Programmable Signal Generator

Introduction

Last week you explored the operation of parallel interfaces in a symbolic way, using lights and switches to demonstrate some kind of real-world activity. Here you will experiment with a system that actually does something useful, a digital function generator that can produce any waveform over a range of frequencies that is user-programmable. Since microprocessor systems are inherently digital and waveforms are inherently analog, the key element in this design is the digital-to-analog converter (DAC). You will use an Analog Devices AD7302, a dual-channel DAC chip located on the SAT board. The DAC must be connected to port PORTD (PD7-PD0), with PORTC 0 and 1 providing extra control signals. PC1 must be set to 0 (to select Channel A, which we will use) --- this should be done once. The circuit that you need to construct is shown below. PC0 must be set to High initially.





The Layout of the ADC chip is shown here.

Use of the DAC is quite simple. When it is given a digital value, the analog correspondent of that value appears on the output (after an appropriate delay for settling, usually a few microseconds). The span of the output voltage is designed to be [0, V_{CC}].

Before the Lab

The DAC requires the following sequence of steps to convert one byte:

1. Send the data to the DAC (i.e. write it to PORTD).
2. Set PC0 to 0 (low) for at least 1 microsecond. You can use a simple delay loop that waits for 1 microsecond.
3. Set PC0 to 1 (high). The TA will help with connecting an oscilloscope to the output of the DAC. You will use this to observe the waveforms.

Write programs that:

1. Loads the series 0x00, 0x01, 0x02 ... 0xFE, 0xFF, 0x00, 0x01 into the DAC. Sketch the output waveform in your notebook. Estimate the period of this waveform. Other commonly used waveforms are the triangle, square and sine wave. Modify the program above so it is useful for generating a triangle wave. It should also be very easy to generate a square wave at the same frequency; do so.
2. Write a program that generates a trapezoidal waveform such as 0x00, 0x01, …, 0xFE, 0xFF, 0xFF, 0xFF, 0xFE, 0xFD, …, 0x00, 0x00. The duration of the constant region in the middle of ascending and descending parts should be variable.
3. A sine wave may be approximated by a series of steps of the appropriate height. To do this, rather than generating the numbers to be sent to the DAC by mathematical progression (as above) you must use a table. There are several approaches to tabular representation of waveforms. A sine wave is, fortunately, symmetric about at least two points. First, the negative portion of a sine wave is identical (but of opposite sign) to the positive portion. Using this characteristic, it is possible to synthesize a 64-point sine waveform with only 32 points. Work out a table of integers that represent the positive half-cycle of a sine wave (remembering you wish to use this to generate voltages across only half the total range of the DAC) and record these in your notebook. These numbers should be representations of the function

   $$\sin((3.1416*k)/32), \text{ for } k = 0,1,2,\ldots,31.$$

   When you have reached the end of this table, you must return to the beginning (table value = 0) and reread the table, negating each value to produce the second (negative) half of the sine wave. This process is then repeated indefinitely. Write a program that generates a sine wave from such a table as rapidly as possible.

   For the more valiant amongst you, a sine wave is also symmetric about its peak; both halves of a half-cycle are mirror images of one another. Try writing a program that synthesizes a 64-point sine wave from only 16 points (you might find it actually requires 17 points....) This is not a formal assignment, but those successful will be awarded a gold star and public adulation.

   There are at least three ways of modifying the frequency generated by your program. (1) Inserting a delay loop between successive loads of the DAC can slow the readout rate down. (2) Skipping points in the number generation (e.g., 2,4,6...) can speed up the

period. (3) The readout rate can be controlled via external interrupts. The next program exercises will explore the first two methods.

1. Generate a triangle wave of variable frequency by counting down a number in a loop between each change of the value sent to the DAC. Predict the alteration of the period of the triangle wave resulting from a change of one count in the delay loop.
2. Increase the frequency of the sine wave by loading every other (or third, or fourth, etc.) point in the table. Write it so that the number of points skipped can be changed.

Test all programs in the simulator. Remember to use the I/O window to check that the bits in the ports are set properly.

In The Lab

Wire up the circuit depicted above. Refer to the datasheet for the IC if needed. Test the operation of the DAC by writing minimum (0) and maximum values into PORTD. Determine the value of Vmax. What affects the value of Vmax?

Run and verify each of your programs. Draw the waveforms actually generated by your DAC, including information on voltage span, any D.C. offset and time in your notebook. Are your predictions of waveform shape and period valid?

Keep notes of your waveforms, frequency and amplitude values, and any modifications you make to your program.